

Step-by-Step GitHub Pages Deployment and Selenium E2E CI/CD Plan for web_platform

This guide outlines the exact, step-by-step actions required to build, deploy, and automate testing for the Vite-based React frontend under the `web_platform/frontend` folder.

Step 1 — Push Your React Project to GitHub

Initialize your Git repository (if not already initialized) at the root of the project (`c:/Users/achyu/Desktop/Projects/CampusConnect`) or the `web_platform` folder, and push to GitHub.

Run the following commands:

```
# Verify you are at the repository root
git init
git add .
git commit -m "Initial upload of CampusConnect project"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/YOUR_REPO.git
git push -u origin main
```

Note: Replace `YOUR_USERNAME` and `YOUR_REPO` with your exact GitHub username and repository name.

Step 2 — Install GitHub Pages Deployment Package

In your terminal, navigate to the frontend directory:

```
cd web_platform/frontend
```

Install the `gh-pages` package as a development dependency:

```
npm install gh-pages --save-dev
```

Step 3 — Configure `package.json` for Deployment

Open `package.json` (file:///c:/Users/achyu/Desktop/Projects/CampusConnect/web_platform/frontend/package.json).

1. Add a `"homepage"` field at the root of the JSON object:

```
"homepage": "https://YOUR_USERNAME.github.io/YOUR_REPO",
```

Replace `YOUR_USERNAME` and `YOUR_REPO` with your repository details.

2. Add deployment scripts inside the `"scripts"` section:

```
"scripts": {
  "dev": "vite",
  "build": "vite build",
  "lint": "eslint . --ext js,jsx --report-unused-disable-directives --max-warnings 0",
  "preview": "vite preview",
  "predeploy": "npm run build",
  "deploy": "gh-pages -d dist"
}
```

[!IMPORTANT] Since this is a Vite project, the build outputs to the `dist` folder, not `build`. Thus, we use `"deploy": "gh-pages -d dist"`.

Step 4 — Configure Vite Base Path for GitHub Pages

Since the site will be hosted at a subpath (https://YOUR_USERNAME.github.io/YOUR_REPO/), Vite needs to prepend the repository name to all asset links.

Open `vite.config.js` (file:///C:/Users/achyu/Desktop/Projects/CampusConnect/web_platform/frontend/vite.config.js) and add the `base` configuration option pointing to your repository name:

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

// https://vitejs.dev/config/
export default defineConfig({
  plugins: [react()],
  base: '/YOUR_REPO/', // Prepend repo name to build assets
  server: {
    port: 5173,
    host: true
  }
})
```

Step 5 — Configure React Router (Switch to HashRouter)

To prevent 404 Page Not Found errors on route refresh or direct link access when hosted on static GitHub Pages, switch from `BrowserRouter` to `HashRouter`.

Open `App.jsx` (file:///C:/Users/achyu/Desktop/Projects/CampusConnect/web_platform/frontend/src/App.jsx).

1. Replace `BrowserRouter` with `HashRouter` in the import on **Line 2**:

```
import { HashRouter as Router, Routes, Route, Navigate, Link, useNavigate, useLocation } from 'react-router-dom';
```

2. The `<Router>` tag on **Lines 134** and **162** will automatically reference `HashRouter` since we aliased it as `Router`.

Step 6 — Deploy the Frontend Application

Build and upload your production application to GitHub Pages by running:

```
npm run deploy
```

This builds your React site into `dist` and uploads it to the `gh-pages` branch on your GitHub repository.

Step 7 — Enable GitHub Pages in Repository Settings

1. Open your repository on GitHub.
2. Go to **Settings** → **Pages** (in the sidebar).
3. Under **Build and deployment**:
 - Set **Source** to **Deploy from branch**.
 - Under **Branch**, select **gh-pages** and click **Save**.
4. Within a few minutes, your site will be live at `https://YOUR_USERNAME.github.io/YOUR_REPO/`.

Step 8 — Add Node.js Selenium E2E Testing Local Setup

Let's create a Node.js-based Selenium testing structure inside the frontend directory.

1. Navigate to the frontend directory:

```
cd web_platform/frontend
```

2. Create a folder named `selenium-tests` and initialize a sub-project:

```
mkdir selenium-tests
cd selenium-tests
npm init -y
```

3. Install Selenium WebDriver and testing dependencies (Mocha and Assert):

```
npm install selenium-webdriver mocha assert --save-dev
```

4. Create a folder structure for tests:

```
mkdir tests
```

5. Create `selenium-tests/tests/login.test.js`:

```

const { Builder, By, until } = require('selenium-webdriver');
const assert = require('assert');

describe('CampusConnect Login E2E Tests', function () {
  this.timeout(30000); // 30s timeout
  let driver;

  before(async function () {
    driver = await new Builder().forBrowser('chrome').build();
  });

  after(async function () {
    if (driver) await driver.quit();
  });

  it('Should login successfully with correct credentials', async function () {
    // Target the live GitHub Pages app (or localhost for local testing)
    await driver.get('https://YOUR_USERNAME.github.io/YOUR_REPO/#/login');

    // Enter email
    const emailInput = await driver.wait(until.elementLocated(By.css("input[type='email']")), 10000);
    await emailInput.sendKeys('sreenu@saveetha.com');

    // Enter password
    const passwordInput = await driver.findElement(By.css("input[type='password']"));
    await passwordInput.sendKeys('Test1234!');

    // Submit
    const submitBtn = await driver.findElement(By.css("button[type='submit']"));
    await submitBtn.click();

    // Verify dashboard redirect (url contains dashboard or similar home elements)
    await driver.wait(until.urlContains('#/'), 15000);
    const currentUrl = await driver.getCurrentUrl();
    assert.ok(currentUrl.includes('#/'), 'Did not redirect to the landing page');
  });
});

```

6. Add a test execution command to selenium-tests/package.json:

```

"scripts": {
  "test": "mocha tests/**/*.test.js"
}

```

Run the tests locally:

```
npm run test
```

Step 9 — Add Stable Elements IDs (Recommended)

To ensure Selenium can select fields reliably even if CSS layouts change, we recommend adding explicit `id` attributes to input fields.

Example inside `AuthScreen.jsx`:

```
<input id="email-input" type="email" ... />
<input id="password-input" type="password" ... />
<button id="login-submit-btn" type="submit" ... />
```

Step 10 — Setup GitHub Actions E2E CI/CD Workflow

To execute these tests automatically on every code push, create a GitHub Actions workflow file.

1. Create the directories at the root of your project:

```
mkdir -p .github/workflows
```

2. Create `.github/workflows/selenium-login.yml`:

```
name: E2E Selenium Testing Pipeline

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  e2e-tests:
    runs-on: ubuntu-latest

    steps:
      - name: Check out Repository
        uses: actions/checkout@v4

      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20

      - name: Install Frontend Dependencies
        run: |
          cd web_platform/frontend
          npm install

      - name: Start Local Mock Server (Optional / For testing localhost)
        # If you need to spin up the local server first, you can run:
        # run: npm run dev &
        run: echo "Testing directly against deployed live environment"

      - name: Set up Chrome & ChromeDriver
        uses: browser-actions/setup-chrome@v1
        with:
          chrome-version: stable

      - name: Install E2E Test Dependencies
        run: |
          cd web_platform/frontend/selenium-tests
          npm install

      - name: Run Headless Selenium Tests
        run: |
          cd web_platform/frontend/selenium-tests
          # Run tests using headless Chrome configuration
          export CHROME_PATH=$(which google-chrome)
          npm run test
```

[!NOTE] In a headless CI environment, Chrome needs to run in headless mode. Modify your `login.test.js` to enable headless mode if executing on GitHub Actions:

```
const chrome = require('selenium-webdriver/chrome');  
let options = new chrome.Options();  
options.addArguments('--headless=new');  
options.addArguments('--no-sandbox');  
options.addArguments('--disable-dev-shm-usage');  
driver = await new Builder().forBrowser('chrome').setChromeOptions(options).build();
```

Step 11 — Push Code and Trigger Pipeline

Commit your changes and push to GitHub:

```
git add .  
git commit -m "Configure GitHub Pages and E2E Selenium CI/CD testing pipeline"  
git push origin main
```

Open the **Actions** tab on your GitHub repository to monitor the live status of your CI/CD test runs.